SAC Room & Equipment Digital Management System

Complete Software Engineering (SE) & Object-Oriented Programming (OOP) Architecture Blueprint • 10 Formal System Diagrams

This engineering specification presents the complete architectural, behavioral, structural, and relational design for digitizing the college **Student Activity Center (SAC) Room and Equipment Register**. Currently, students must manually write their name, roll number, mobile number, equipment name, sport category, and arrival/departure timestamps into a physical paper register at the SAC counter, with zero visibility into whether a room (such as the Dance Room or Music Room) or sports equipment (such as Table Tennis rackets, balls, Tennis rackets, Chess sets, or Carrom boards) is currently available before walking across campus.

The system replaces the paper register with a real-time, role-segregated web application backed by an Object-Oriented domain model, a server-enforced **2-minute issue request countdown**, an automated **3-strikes-in-1-hour (→ 5-hour cooldown) anti-spam policy**, **peer-to-peer equipment/room transfers** with atomic register row splitting, and **state-linked transfer invalidation** on guard return.

Diagram ID	Engineering Diagram Title	SE / OOP Category	Core Architectural Mechanics Covered
Diagram 1	Three-Tier System Context & Real-Time Architecture	System Architecture (SE)	Role-based clients (Student, Guard, Admin), Google OAuth verifier, OOP Domain Engine, WebSocket Hub, Serverless PostgreSQL
Diagram 2	UML Use Case Model & Actor Boundary	Requirements Engineering (SE)	5 Actors, 16 Use Cases, **`<<include>>`** & **`<<extend>>`** stereotypes across checkout, transfer, return, and admin governance
Diagram 3	Object-Oriented UML Class Diagram	Structural OOP Design (OOD)	Abstract **`User`** & **`SACItem`** inheritance trees, polymorphism, **`IssueRequest`**, **`RegisterEntry`**, **`PenaltyPolicy`**, and **`RealtimeEventHub`**
Diagram 4	Entity-Relationship (ER) & PostgreSQL Schema	Database Engineering (SE)	Crow's-foot schema for **`users`**, **`sac_items`**, **`issue_requests`**, and **`register_logs`** (1 row/unit + **`batch_id`** + **`pending_transfer_to_id`**)

Diagram ID	Engineering Diagram Title	SE / OOP Category	Core Architectural Mechanics Covered
Diagram 5	UML Sequence Diagram A: Issue, 2m TTL & Cooldown	Behavioral Interaction (OOP/SE)	2-step confirmation, 120s server TTL countdown, concurrent request race handling, and >3 timeouts/1h → 5h penalty lock
Diagram 6	UML Sequence Diagram B: Peer Transfer & Return	Behavioral Interaction (OOP/SE)	Atomic peer transfer split (A closed + B opened), guard return wiping pending transfers, and damaged item Defaulter flagging
Diagram 7	UML Statechart (Finite State Machine) Diagrams	State Modeling (OOP/SE)	Coupled lifecycles for **`IssueRequest`**, **`RegisterEntry`** (with state-linked transfer kill), and **`Student`** account eligibility
Diagram 8	Multi-Actor UML Activity & Swimlane Workflow	Process Modeling (SE)	4 parallel swimlanes (Student A, Backend Engine, SAC Guard/Admin, Student B) with decision guards and state branches
Diagram 9	Data Flow Diagrams (DFD Level 0 & Level 1)	Structured Analysis (SE)	Context-level boundary (0.0) and expanded processes (1.0–6.0) interacting with persistent stores (D1–D4)
Diagram 10	OOP Package Structure & GoF Design Patterns	Design Patterns (OOP)	5-layer package hierarchy and 4 Gang-of-Four patterns (Template Method, State, Strategy, and Observer)

1. Three-Tier System Context & Real-Time Architecture Replaces Manual Register bottlenecks

The system follows a strict three-tier client-server architecture augmented by a persistent bidirectional WebSocket layer. Students, SAC Guards, and Administrators interact with role-specific interfaces tailored to their operational speed requirements. Because the SAC counter is compact and high-traffic during evening hours, the Student client handles item selection and double-confirmation, while the SAC Guard client operates as a zero-typing, single-tap approval and return terminal.

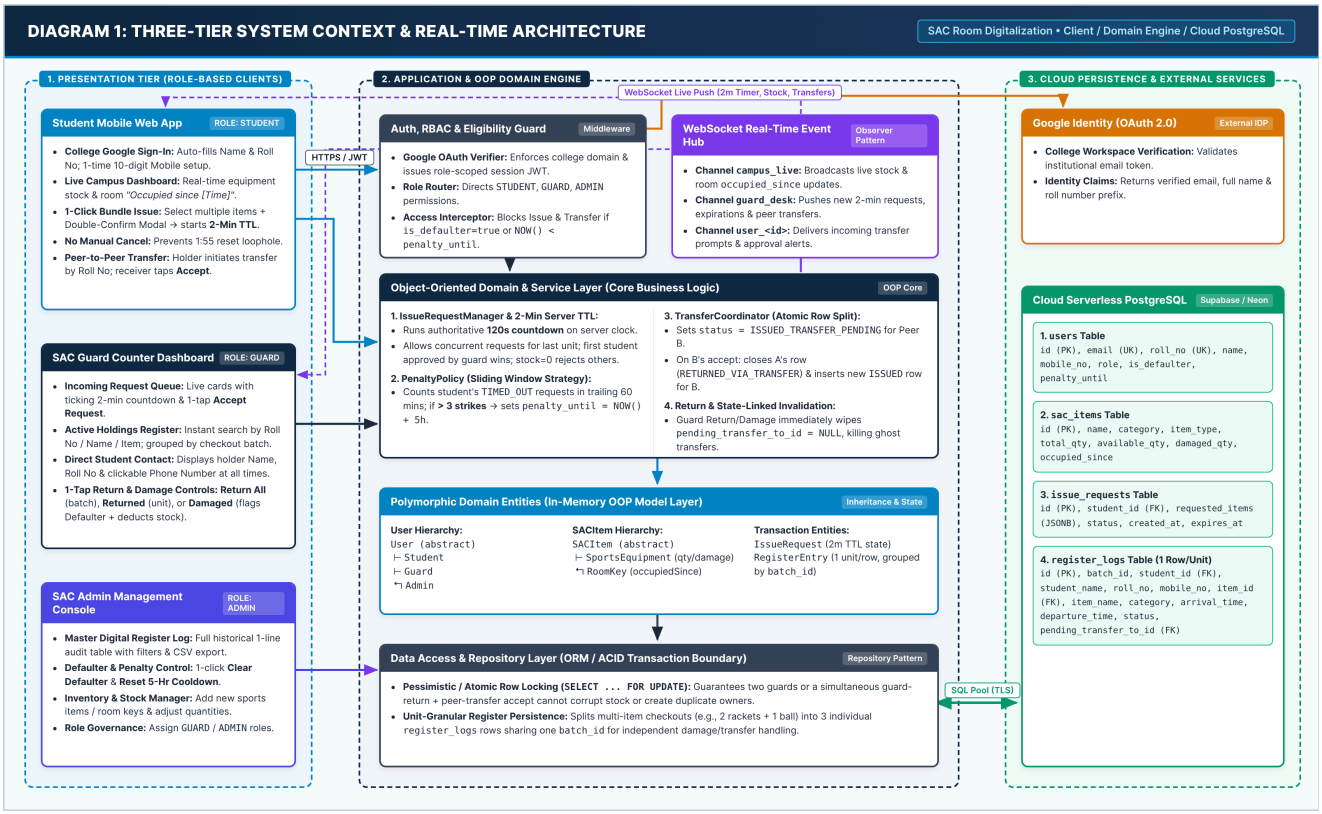


Figure 1: Three-Tier System Context and Real-Time WebSocket Architecture connecting Role Clients, the OOP Domain Engine, and Cloud PostgreSQL.

- **Presentation Tier (Role-Segregated Clients):** Routes authenticated users based on the **`users.role`** column to the **Student Mobile Web App** (live campus inventory, 2-step issue request, peer transfer), the **SAC Guard Counter Dashboard** (live 2-minute incoming request queue, active holdings grouped by batch with clickable student mobile numbers, and 1-tap *Accept* / *Returned* / *Damaged* controls), or the **SAC Admin Console** (master register table, CSV export, Defaulter clearance, and stock management).
- **Application & OOP Domain Engine:** Enforces authentication via college Google OAuth 2.0, runs the authoritative **120-second server-side TTL timer** (preventing students from bypassing timeouts by manipulating their device clock), evaluates the trailing 1-hour timeout window, and coordinates atomic database transactions.
- **Cloud Persistence Tier (Serverless PostgreSQL):** Uses row-level locking (**`SELECT ... FOR UPDATE`**) on **`sac_items`** and **`register_logs`** so concurrent approvals or simultaneous guard-return and peer-transfer clicks never corrupt stock counts or ownership records.

2. UML Use Case Model Defines Role Boundaries & Automated Policy Triggers

The functional scope is partitioned across five actors: three human actors (**Student**, **SAC Guard**, and **SAC Admin**) and two system/external actors (**Google OAuth IDP** and the **Server TTL Scheduler**). Mandatory pre-conditions and post-actions are modeled using UML **`<<include>>`** dependencies, while conditional penalty and damage branches are modeled via **`<<extend>>`** dependencies.

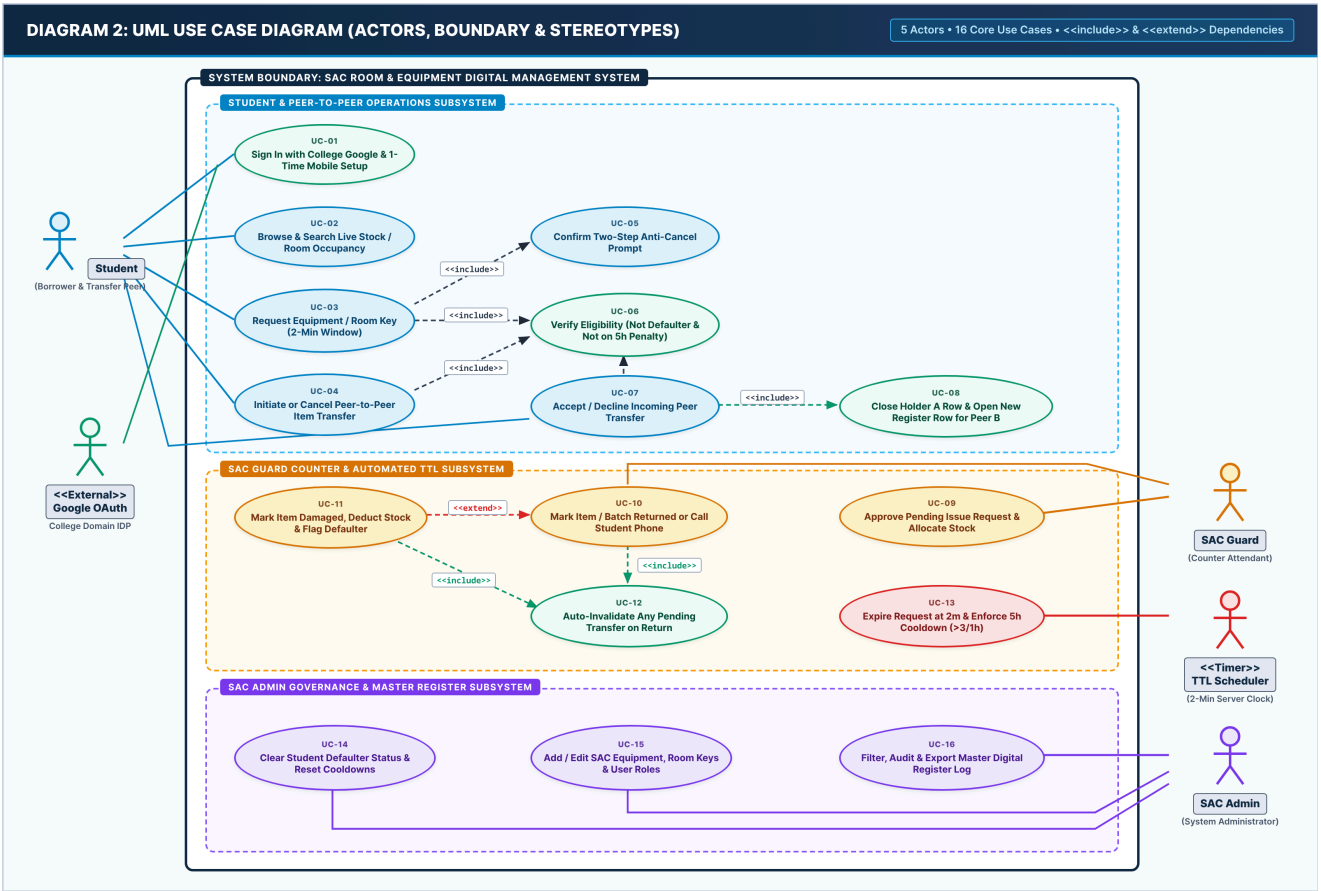


Figure 2: UML Use Case Diagram illustrating 5 Actors, 16 Use Cases, and formal <<include>> and <<extend>> relationships across three operational subsystems.

UC ID	Use Case Name	Primary Actor	Included / Extended Rules & Post-Conditions
UC-01	Sign In & 1-Time Mobile Setup	Student, Google IDP	Verifies college Google email, extracts Name & Roll No, prompts once for 10-digit mobile number.
UC-03	Request Equipment / Room Key	Student	**<<include>> UC-05** (2-Step Anti-Cancel Prompt) & **UC-06** (Eligibility Check). Starts 2-min timer; manual cancel disabled.
UC-04 / 07	Initiate & Accept Peer Transfer	Student A, Student B	**<<include>> UC-06** (Verify Peer B is not Defaulter/Cooldown) & **UC-08** (Close A's register row & insert B's row).
UC-09	Approve Pending Issue Request	SAC Guard	Approves request if **`t < 120s`** and **`available_qty >= req_qty`**; creates 1 **`register_logs`** row per unit sharing a **`batch_id`**.
UC-10 / 12	Mark Item / Batch Returned	SAC Guard	**<<include>> UC-12**: Sets **`status='RETURNED'`**, **`departure_time=NOW()`**, and wipes **`pending_transfer_to_id=NULL`**.

UC ID	Use Case Name	Primary Actor	Included / Extended Rules & Post-Conditions
UC-11	Mark Item Damaged & Flag Defaulter	SAC Guard	**<<extend>> UC-10**: Increments **damaged_qty**, sets **status='DAMAGED'**, and sets **users.is_defaulter=TRUE**.
UC-13	Expire Request & Impose Cooldown	TTL Scheduler	Marks unapproved request **TIMED_OUT** at 120s; if student has **>3 TIMED_OUT** in trailing 1h, sets **penalty_until=NOW()+5h**.
UC-14–16	Admin Governance & Register Audit	SAC Admin	Clears **is_defaulter**, resets 5h cooldown, adds/edits inventory stock & roles, and exports master register logs.

3. Object-Oriented UML Class Hierarchy Encapsulates Polymorphic Inventory & Roles

To satisfy rigorous Object-Oriented Design principles, the domain layer avoids procedural scripts and models every participant and physical asset as an encapsulated class with explicit visibility modifiers (**-** private, **#** protected, **+** public). Two primary inheritance hierarchies—**User** and **SACItem**—leverage polymorphism to handle role-specific permissions and asset-specific allocation rules cleanly.

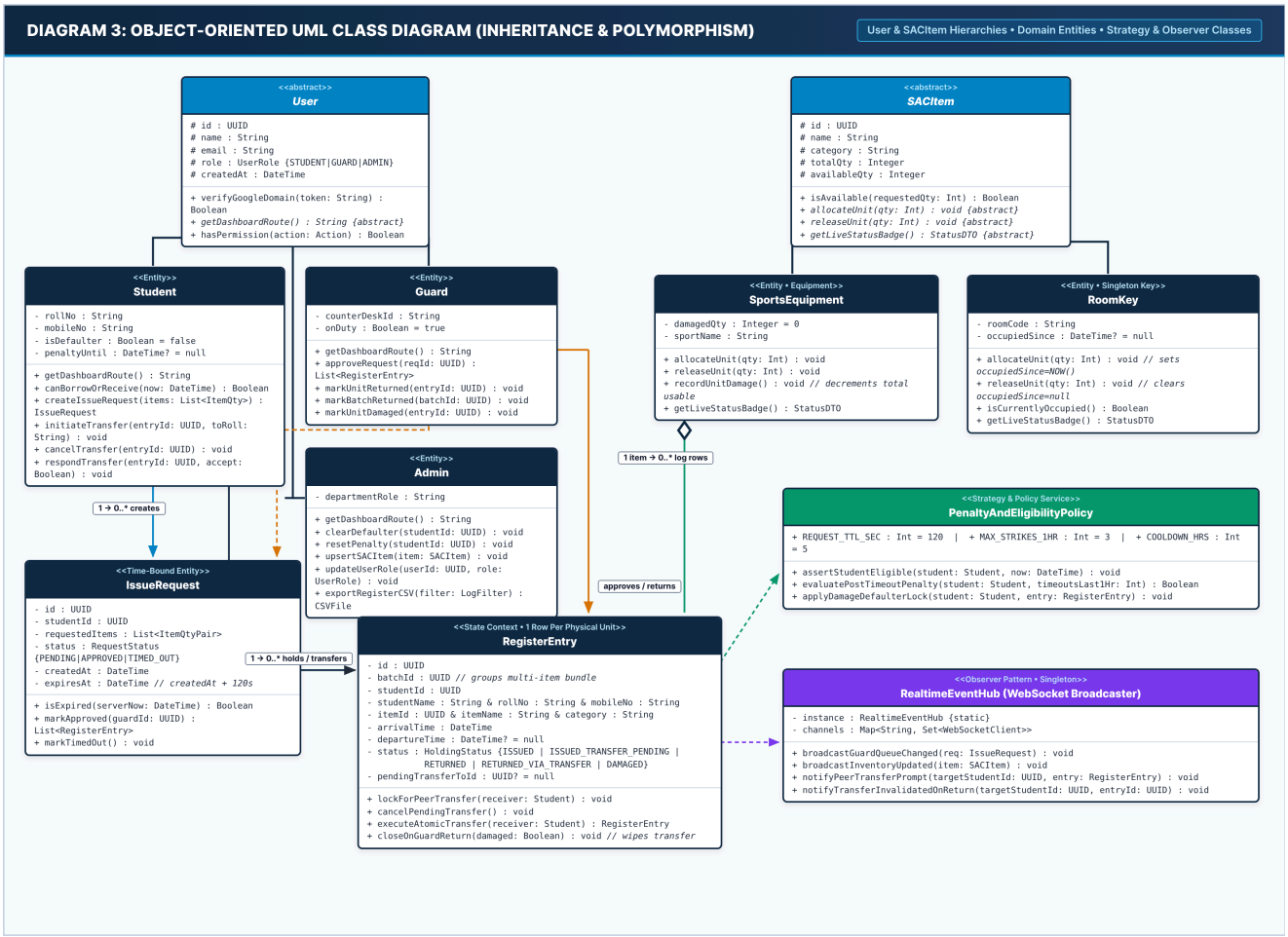


Figure 3: Full 3-Compartment UML Class Diagram showing Inheritance (User, SACItem), State Context (RegisterEntry), Strategy (PenaltyPolicy), and Observer (RealtimeEventHub).

- **Polymorphic `SACItem` Hierarchy:** Both **`SportsEquipment`** (e.g., 10 TT Rackets, 20 TT Balls) and **`RoomKey`** (e.g., Dance Room Key, Music Room Key) inherit from the abstract base class **`SACItem`**. When a guard approves a checkout or processes a return, the service invokes **`item.allocateUnit(qty)`** or **`item.releaseUnit(qty)`** polymorphically: **`SportsEquipment`** adjusts multi-unit counts and tracks **`damagedQty`**, whereas **`RoomKey`** enforces singleton capacity (**`totalQty = 1`**) and stamps or clears **`occupiedSince`** so students across campus see *"Occupied since [Time]"*.
- **Granular Unit-Level `RegisterEntry` Class:** When a student checks out a bundle (such as 2 TT Rackets and 1 TT Ball), **`Guard.approveRequest()`** instantiates three distinct **`RegisterEntry`** objects sharing a common **`batchId`**. This allows the guard UI to offer a 1-tap **`markBatchReturned(batchId)`** button for normal returns while allowing a single broken TT ball to be marked **`DAMAGED`** (or a single racket to be transferred to a friend) without corrupting the remaining items in the bundle.

4. Relational Database Schema Embeds Transfer State Directly in Register Rows

The physical PostgreSQL schema uses four normalized tables with strategic denormalization on **`register_logs`** (`student_name`, `roll_no`, `mobile_no`, `item_name`, `category`) so the digital register table renders and exports as a single-table scan identical to the physical SAC register book.

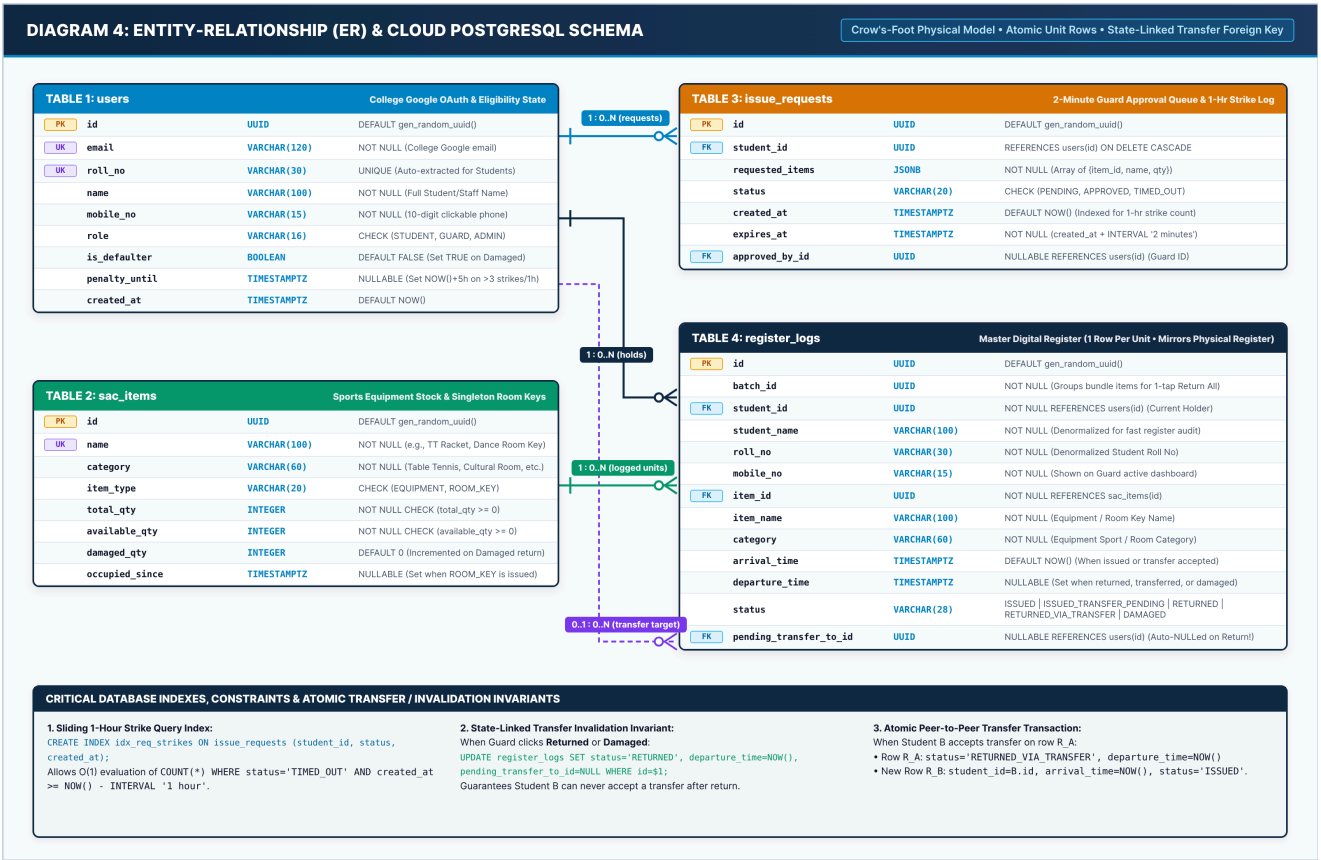


Figure 4: Crow's-Foot Entity-Relationship (ER) Diagram and Physical PostgreSQL Schema with Atomic Transfer and Invalidation Invariants.

Crucially, rather than storing peer-to-peer transfer requests in a detached table that could become out-of-sync when a student returns an item to the guard counter, the pending transfer target is stored directly on the active holding row via **`register_logs.pending_transfer_to_id`** alongside **`status = 'ISSUED_TRANSFER_PENDING'`**. Consequently, when the guard clicks **Returned** or **Damaged**, the SQL update statement sets **`status = 'RETURNED'`** (or **`'DAMAGED'`**), **`departure_time = NOW()`**, and

`pending_transfer_to_id = NULL` in a single atomic row operation—making it impossible for a stale transfer request to survive after an item has been returned to the SAC counter.

5. Sequence Diagram A: Server-Authoritative 2-Minute TTL & Sliding-Window Penalty

To prevent students from booking scarce items from their hostel rooms while walking slowly to the SAC building, the issue workflow combines a two-step non-cancelable confirmation prompt with a strict 120-second server timer and a sliding 1-hour strike counter. Multiple students may request the last available unit simultaneously; the guard approves whichever student physically arrives at the SAC counter first.

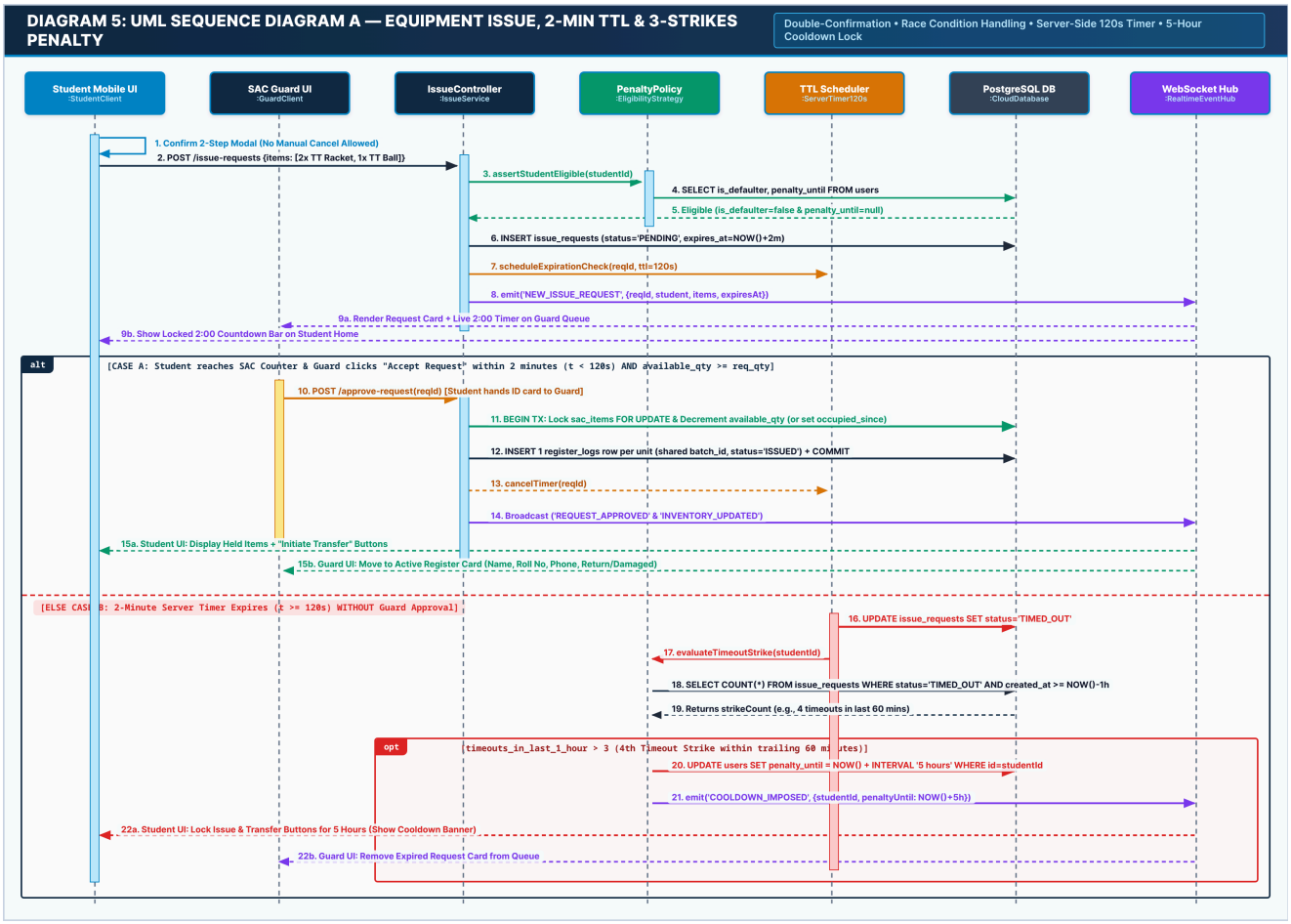


Figure 5: UML Sequence Diagram A detailing the 2-Step Confirmation, 2-Minute Server TTL Countdown, Guard Approval Transaction, and 3-Strikes/5-Hour Cooldown Enforcement.

Interaction Phase	Participating Objects	Technical Execution & Concurrency Guarantee
Steps 1–5: Pre-Flight Eligibility	`StudentClient` → `IssueService` → `PenaltyPolicy`	After the student confirms the 2-step modal, `PenaltyPolicy` queries `users` to verify `is_defaulter = FALSE` and `(penalty_until IS NULL OR penalty_until <= NOW())`.
Steps 6–9: 120s Timer Registration	`IssueService` → `TTLScheduler` → `WebSocketHub`	Inserts `issue_requests` with `status='PENDING'` and `expires_at = NOW() + 120s`, schedules a 120s callback, and broadcasts the live countdown card to both Guard and Student screens.

Interaction Phase	Participating Objects	Technical Execution & Concurrency Guarantee
Steps 10–15: Guard Approval (Case A)	`GuardClient` → `IssueService` → `PostgreSQL`	Executes `SELECT ... FOR UPDATE` on `sac_items`, verifies `available_qty >= req_qty`, decrements stock (or sets `occupied_since=NOW()`), inserts 1 `register_logs` row per unit with shared `batch_id`, and cancels the timer.
Steps 16–22: Timeout & Penalty (Case B)	`TTLScheduler` → `PenaltyPolicy` → `PostgreSQL`	At `t = 120s`, updates request to `TIMED_OUT` and counts trailing 60-minute timeouts for that student. If `strikeCount > 3`, updates `users.penalty_until = NOW() + INTERVAL '5 hours'` and locks the student's UI.

6. Sequence Diagram B: Peer-to-Peer Transfer & State-Linked Return Invalidation

When Student A holds an item (such as the Dance Room Key or a TT Racket) and wishes to return to their hostel while Student B continues using the facility, Student A initiates a peer transfer to Student B. The guard is not involved in approving the transfer, but the guard's active dashboard automatically updates to display Student B's name, roll number, and clickable phone number the instant Student B accepts.

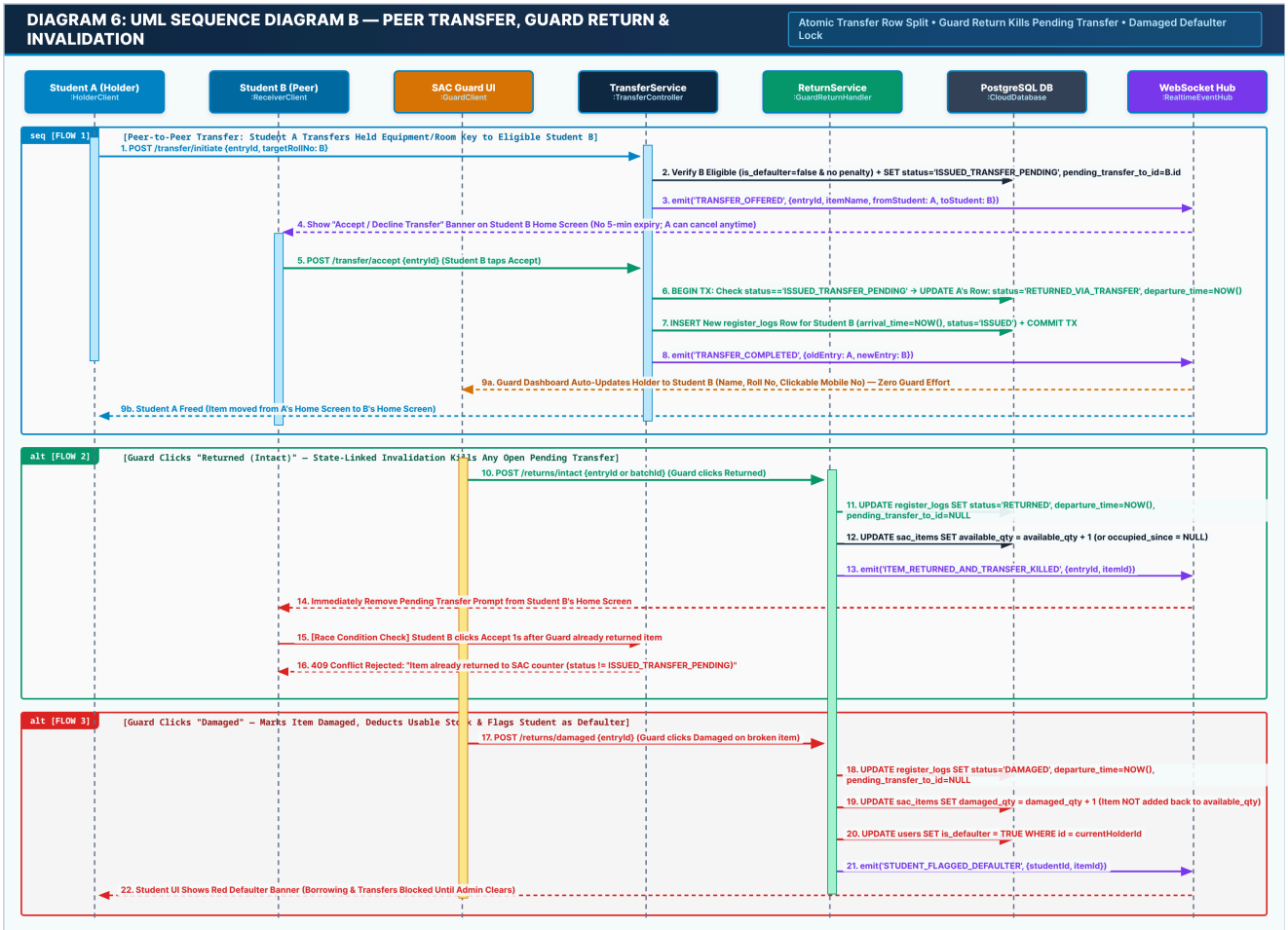


Figure 6: UML Sequence Diagram B illustrating Atomic Peer Transfer Row Splitting (Flow 1), Guard Return Killing Pending Transfers (Flow 2), and Damaged Item Defaulter Locking (Flow 3).

- **Flow 1 — Atomic Peer-to-Peer Transfer Row Split:** When Student B taps **Accept Transfer**, `TransferService` runs an ACID transaction that verifies the row is still in `ISSUED_TRANSFER_PENDING`, closes Student A's register row (`status = 'RETURNED_VIA_TRANSFER'`, `departure_time = NOW()`, `pending_transfer_to_id = NULL`), and inserts a new register row for Student B (`status = 'ISSUED'`, `arrival_time = NOW()`).
- **Flow 2 — State-Linked Invalidation on Guard Return:** Because transfer requests do not use an arbitrary 5-minute expiry, Student A could initiate a transfer to B at `t = 0` and then hand the item back to the guard at `t = 1 min`. When the guard clicks **Returned**, `ReturnService` sets `status = 'RETURNED'` and `pending_transfer_to_id = NULL`, immediately removing the prompt from B's screen and rejecting any race-condition click by B with `HTTP 409 Conflict`.
- **Flow 3 — Damaged Return & Defaulter Enforcement:** When the guard clicks **Damaged** on a returned unit, `ReturnService` transitions the row to `DAMAGED`, increments `sac_items.damaged_qty` (leaving `available_qty` unincremented so broken gear is removed from circulation), and sets `users.is_defaulter = TRUE` on the current holder.

7. Coupled UML Statechart Diagrams Govern Entity & Account Lifecycles

The behavioral correctness of the SAC system relies on three coupled Finite State Machines (FSMs): the short-lived `IssueRequest` state machine, the unit-level `RegisterEntry` holding state machine, and the long-lived `Student` account eligibility state machine.

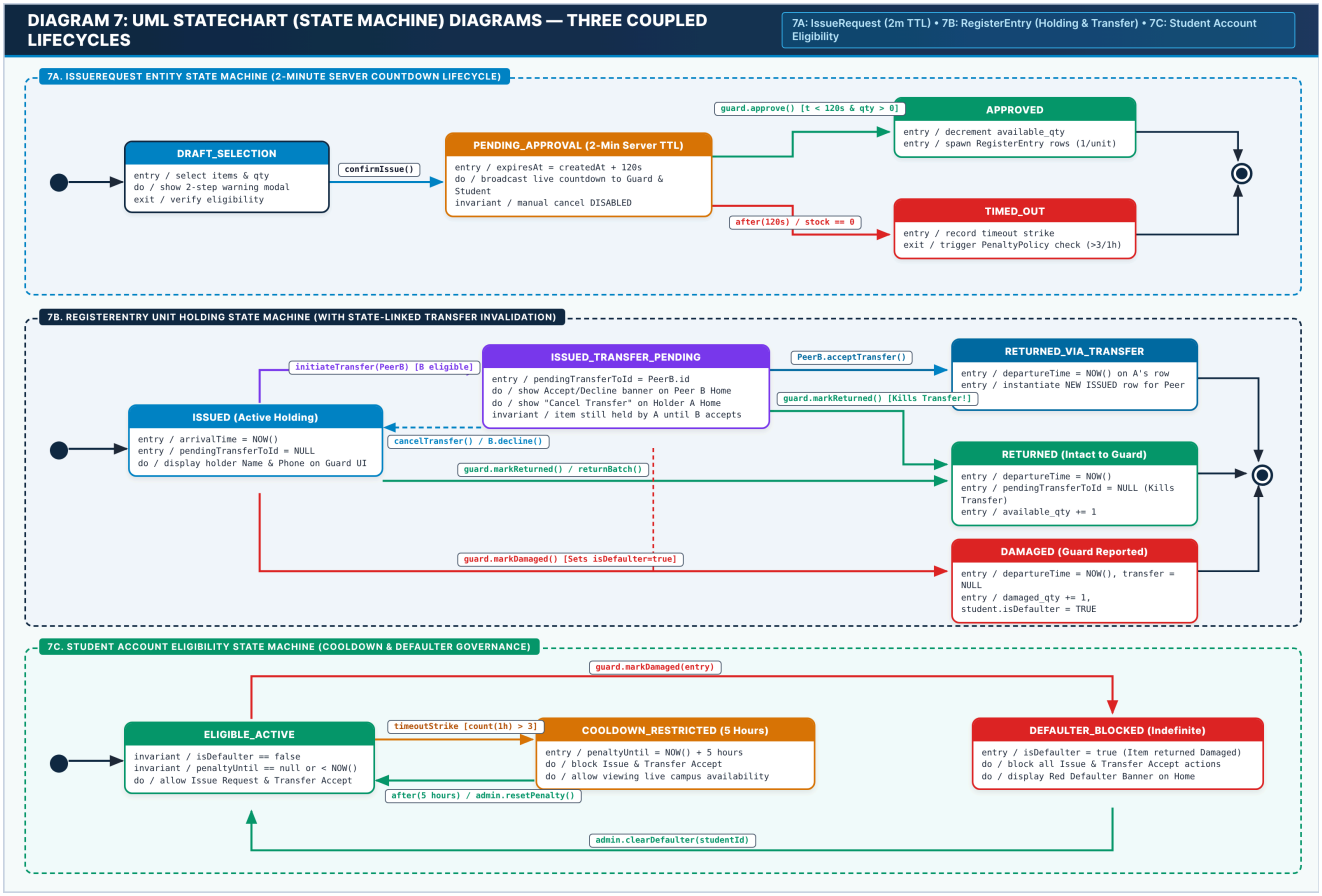


Figure 7: Coupled UML Statechart Diagrams for IssueRequest (7A), RegisterEntry Holding & Transfer States (7B), and Student Account Eligibility (7C).

State Machine	Source State → Target State	Trigger Event & Guard Condition	Atomic Transition Action (Side Effect)
7A. IssueRequest	`PENDING_APPROVAL` → `APPROVED`	`guard.approve() [t < 120s && stock > 0]`	Cancel 120s timer; decrement stock; spawn `RegisterEntry` rows.
7A. IssueRequest	`PENDING_APPROVAL` → `TIMED_OUT`	`after(120s)` or `[stock == 0]`	Record timeout strike; invoke `PenaltyPolicy.evaluate()`.
7B. RegisterEntry	`ISSUED` → `ISSUED_TRANSFER_PENDING`	`A.initiateTransfer(B) [B.canBorrow()]`	Set `pendingTransferToId = B.id`; push prompt to Student B.
7B. RegisterEntry	`ISSUED_TRANSFER_PENDING` → `RETURNED_VIA_TRANSFER`	`B.acceptTransfer() [status == PENDING]`	Set `departureTime = NOW()` on A's row; insert new `ISSUED` row for B.
7B. RegisterEntry	`ISSUED / PENDING` → `RETURNED`	`guard.markReturned()`	Set `departureTime = NOW()`, `pendingTransferToId = NULL`, `available_qty + 1`.
7B. RegisterEntry	`ISSUED / PENDING` → `DAMAGED`	`guard.markDamaged()`	Set `departureTime = NOW()`, `damaged_qty + 1`, `holder.isDefaulter = TRUE`.
7C. Student	`ELIGIBLE_ACTIVE` → `COOLDOWN_RESTRICTED`	`timeoutStrike [timeouts_in_1h > 3]`	Set `penaltyUntil = NOW() + 5 hours`; block Issue & Transfer Accept.
7C. Student	`ANY` → `DEFAULTER_BLOCKED`	`guard.markDamaged(entry)`	Set `isDefaulter = TRUE`; block borrowing until `admin.clearDefaulter()`.

8. Multi-Actor Activity Swimlanes Coordinate Student, Guard & Peer Actions

While Sequence Diagrams focus on object-level message passing, the UML Activity Swimlane Diagram maps the end-to-end operational control flow across the four concurrent participants: **Student A (Primary Borrower)**, the **SAC Backend & Timer Engine**, the **SAC Guard / Admin**, and **Student B (Transfer Peer)**. Each vertical partition isolates the exact decisions and responsibilities owned by that actor:

- **Swimlane 1 (Student A — Primary Borrower):** Initiates Google OAuth login, selects equipment quantities or room keys from the live campus feed, confirms the two-step non-cancelable prompt, collects approved gear at the SAC desk, and optionally initiates a peer transfer to Student B before leaving for the hostel.
- **Swimlane 2 (SAC Backend & Timer Engine):** Enforces pre-checkout eligibility guards, runs the authoritative 120-second countdown, applies the 5-hour cooldown penalty on the 4th timeout within 1 hour, executes atomic register row splits during peer transfers, and wipes pending transfers on guard return.

- **Swimlane 3 & Swimlane 4 (SAC Guard / Admin & Student B):** The Guard approves arrivals within 2 minutes and inspects returned items (marking them *Returned* or *Damaged*), while Student B accepts incoming peer transfers to become the new active holder on the guard dashboard.

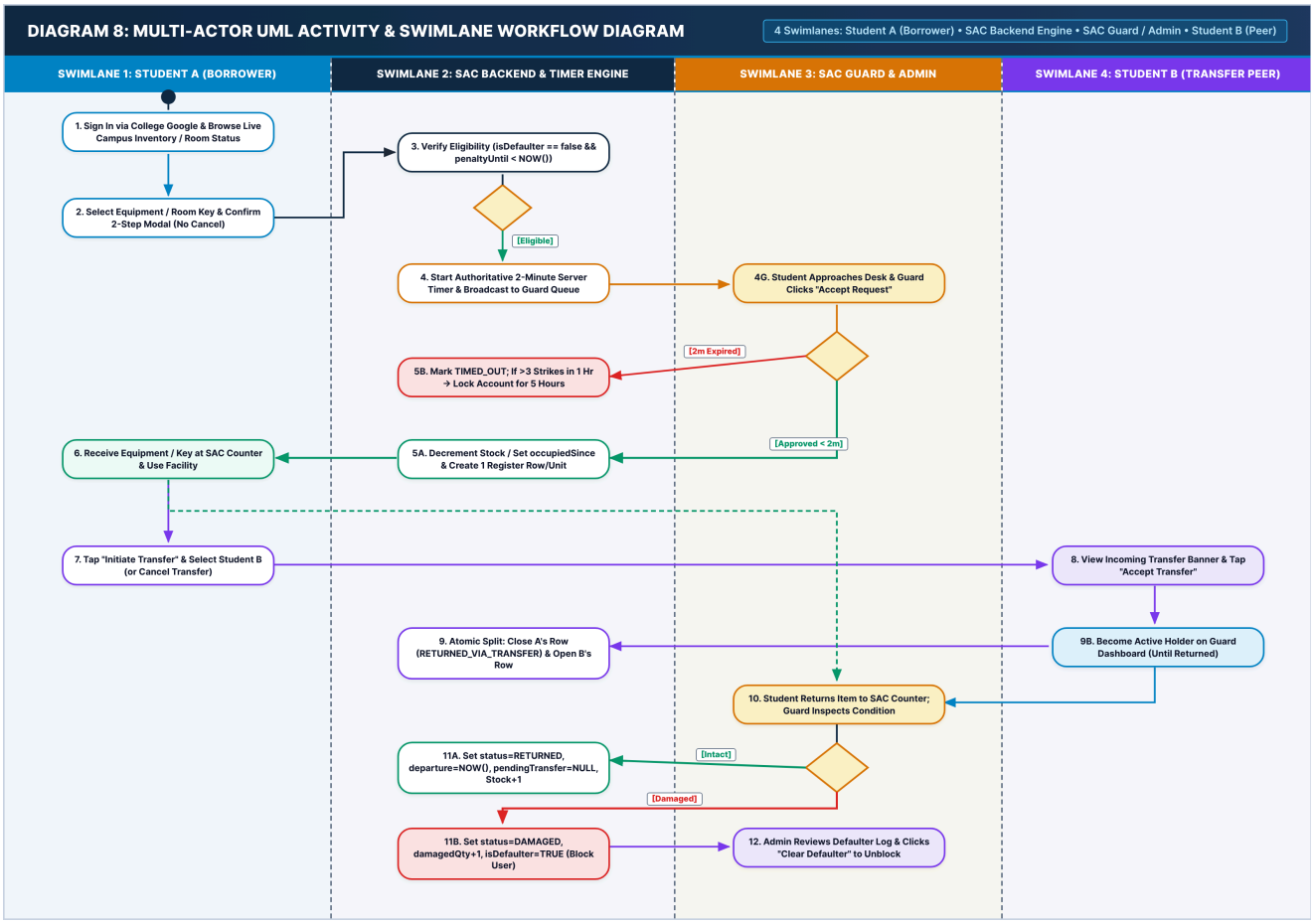


Figure 8: Four-Swimlane UML Activity Diagram tracing decision diamonds, timer expirations, peer transfers, and guard inspection branches.

Notice how the swimlane structure highlights the operational autonomy of the system: once the SAC Guard clicks **Accept Request** in Swimlane 3 (Step 4G), all subsequent peer-to-peer handoffs between Student A (Swimlane 1) and Student B (Swimlane 4) are orchestrated entirely by the Backend Engine (Swimlane 2, Step 9) without interrupting the guard at the counter. The guard only re-enters the workflow at Step 10 when the final holder physically brings the equipment or room key back to the SAC desk.

9. Data Flow Diagrams (DFD Level 0 & Level 1) Map Process-to-Store Boundaries

For structured Software Engineering analysis, Figure 9 presents both the **Level 0 Context DFD** (defining external entity data streams into and out of Process **`0.0`**) and the **Level 1 Expanded DFD** decomposing the system into six cohesive functional processes (**`1.0`** through **`6.0`**) and four relational data stores (**`D1: users`**, **`D2: sac_items`**, **`D3: issue_requests`**, and **`D4: register_logs`**).

- **Level 0 Context Boundary (Process 0.0):** Captures all external inputs and outputs crossing the system boundary between Students (search queries, issue bundles, transfer requests), SAC Guards (2-minute queue approvals, intact/damaged returns), SAC Admins (stock updates, Defaulter clearances, CSV exports), and the College Google OAuth Provider.

- **Level 1 Data Store Isolation (D1–D4):** Ensures every persistent table is accessed only through dedicated domain processes—**`D1 (users)`** stores role and eligibility state, **`D2 (sac_items)`** tracks stock and **`occupied_since`** timestamps, **`D3 (issue_requests)`** logs 2-minute TTL requests, and **`D4 (register_logs)`** maintains the unit-level audit trail.

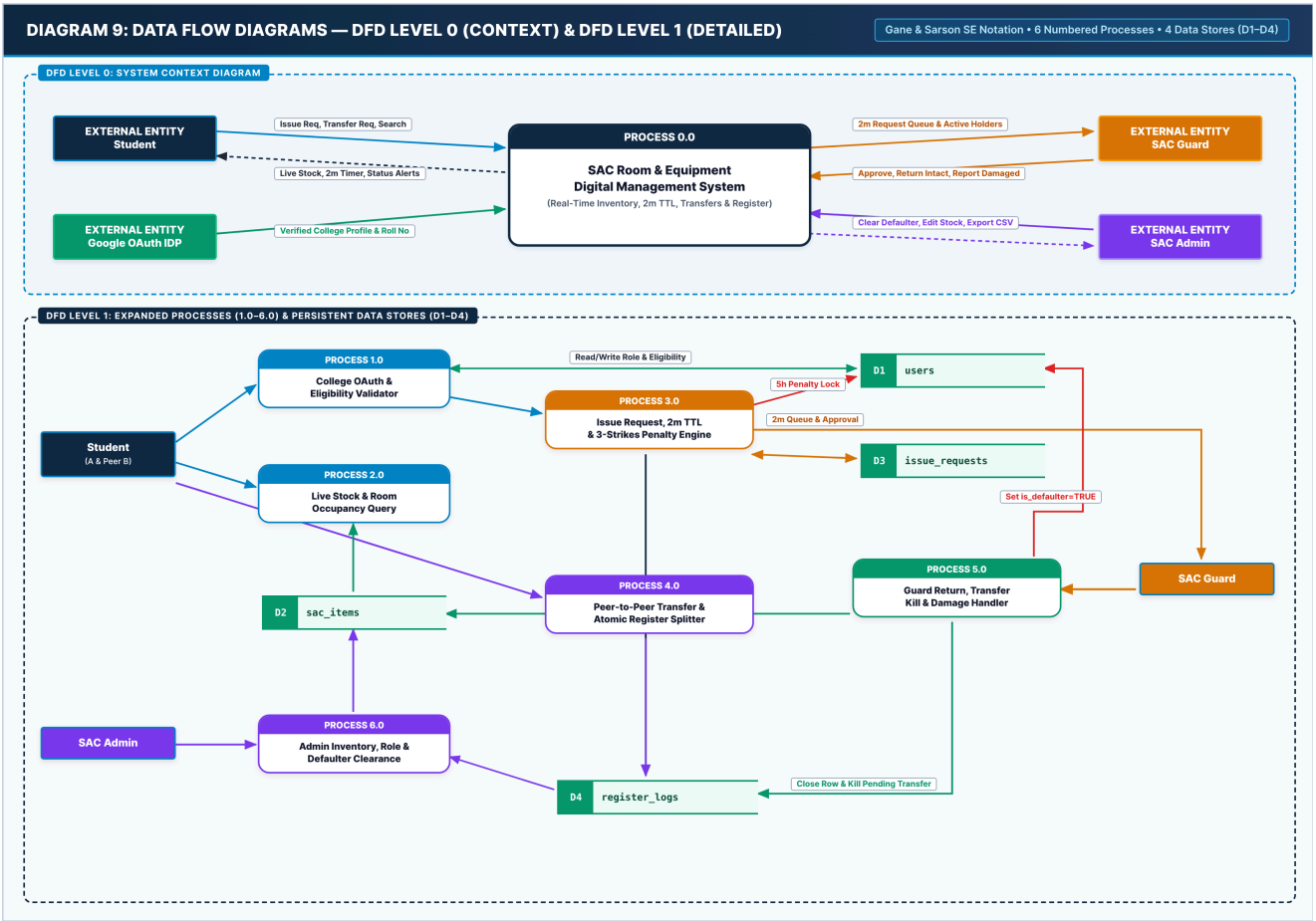


Figure 9: Gane & Sarson Data Flow Diagrams — Level 0 Context Diagram (Top) and Level 1 Detailed Process & Data Store Breakdown (Bottom).

- **Process 1.0 & Process 2.0 (Authentication & Live Query):** Process **`1.0`** validates college OAuth tokens and reads/writes eligibility state in **`D1 (users)`**, while Process **`2.0`** serves non-blocking campus-wide reads from **`D2 (sac_items)`** even for students who are currently restricted from borrowing.
- **Process 3.0, 4.0 & 5.0 (Core Transaction Triad):** Process **`3.0`** manages **`D3 (issue_requests)`** and writes initial holdings to **`D4 (register_logs)`**; Process **`4.0`** executes atomic transfer splits inside **`D4`**; and Process **`5.0`** closes holdings in **`D4`**, restores stock in **`D2`**, and updates Defaulter flags in **`D1`**.

10. Layered OOP Package Architecture & Gang-of-Four (GoF) Design Patterns

To ensure the codebase remains modular, testable, and language-portable (whether implemented in TypeScript/Node, Java Spring Boot, or Python), the system is organized into five strictly unidirectional packages and implements four foundational Gang-of-Four (GoF) Object-Oriented Design Patterns:

- **Unidirectional Package Layering:** Upper layers (**`presentation.dashboards`** and **`api.controllers`**) depend downward on **`domain.services`** and **`domain.entities`**, while **`infrastructure.persistence`** encapsulates SQL queries and WebSocket sockets behind clean repository and event interfaces.

- **Decoupled Domain Logic:** By keeping **`User`**, **`SACItem`**, **`IssueRequest`**, and **`RegisterEntry`** inside a pure OOP domain package free of HTTP or SQL boilerplate, all state transitions and penalty rules can be verified via unit tests before connecting to the cloud database.

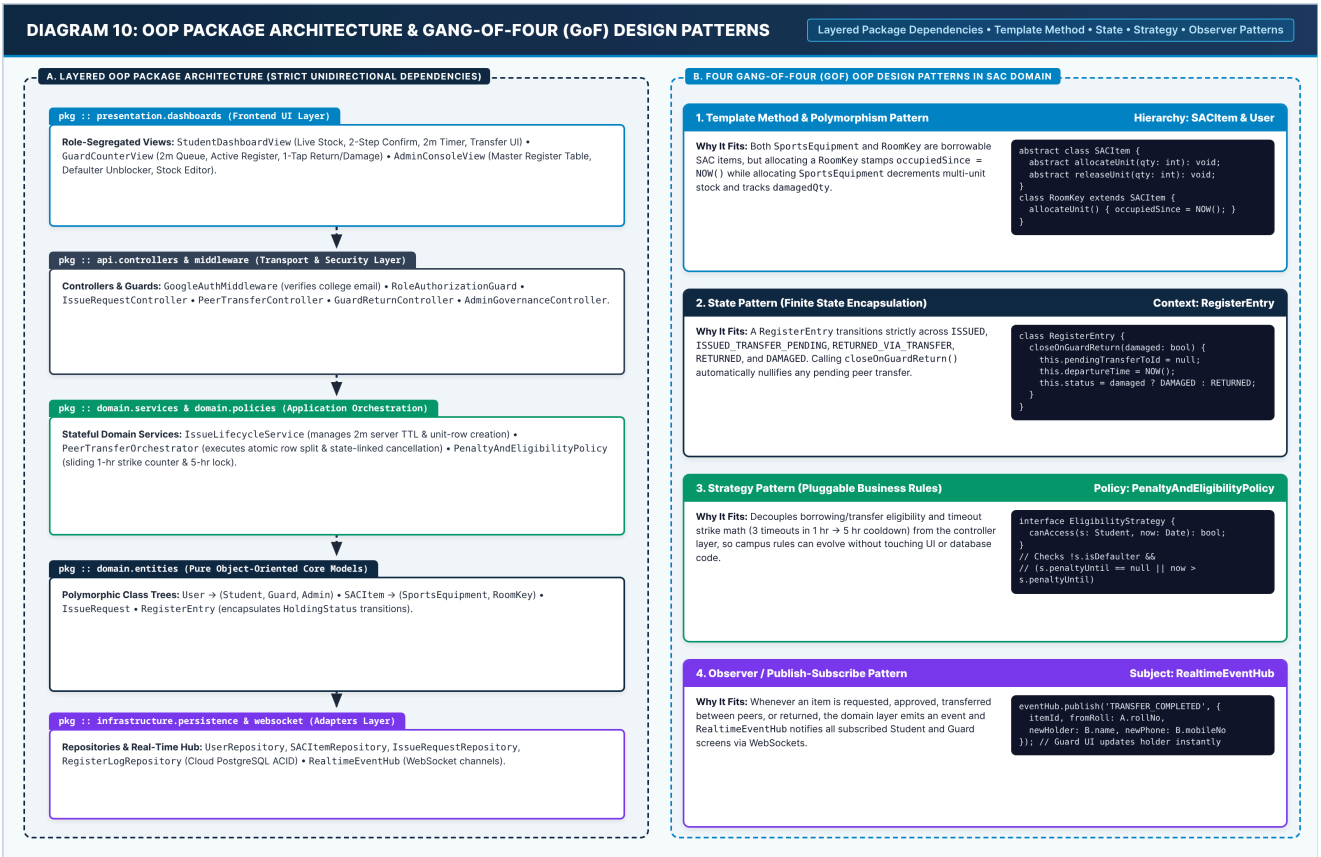


Figure 10: Five-Layer OOP Package Dependency Graph (Left) and Structural Implementation of Four Gang-of-Four Design Patterns (Right).

GoF Design Pattern	Target Domain Classes	Architectural Justification & Maintenance Benefit
1. Template Method & Polymorphism	**`SACItem`** → **`SportsEquipment`**, **`RoomKey`**; **`User`** → **`Student`**, **`Guard`**, **`Admin`**	Defines invariant checkout/return skeletons in **`SACItem`** while deferring quantity vs. **`occupiedSince`** timestamp mutations to concrete subclasses.
2. State Pattern	**`RegisterEntry`** & **`HoldingStatus`**	Encapsulates legal state transitions and guarantees that invoking **`closeOnGuardReturn()`** from either **`ISSUED`** or **`ISSUED_TRANSFER_PENDING`** nullifies pending transfers.
3. Strategy Pattern	**`PenaltyAndEligibilityPolicy`**	Isolates the 120s TTL, 3-strikes-in-1-hour window, and 5-hour cooldown calculation into a pluggable policy class that can be unit-tested independently of the database.
4. Observer Pattern	**`RealtimeEventHub`** (WebSocket Singleton)	Decouples domain state mutations from UI updates by broadcasting domain events (**`NEW_ISSUE_REQUEST`**, **`TRANSFER_COMPLETED`**, **`ITEM_RETURNED`**) to subscribed clients.